

РУКОВОДСТВО ПРОГРАММИСТА

1. НАЗНАЧЕНИЕ И УСЛОВИЯ ПРИМЕНЕНИЯ ПРОГРАММЫ

Настоящее руководство предназначено для квалифицированных специалистов (программистов, системных инженеров и сотрудников технической поддержки), осуществляющих развертывание, администрирование, модификацию и последующее масштабирование мобильного приложения «Книжный трекер». Документ содержит исчерпывающее описание программной архитектуры, порядка конфигурирования облачных сервисов и сборки конечного дистрибутива.

1.1. Требования к программному окружению и рабочему месту разработчика

Для ведения разработки, отладки и последующей компиляции программного продукта рабочее место программиста должно быть укомплектовано следующим программным обеспечением:

- **Среда редактирования исходного кода:** Интегрированная среда разработки (IDE) *Visual Studio Code* (версии 1.80 и выше) с установленными расширениями для подсветки синтаксиса веб-технологий (HTML, CSS, JavaScript).
- **Среда локального тестирования и отладки:** Современный веб-браузер на базе движка Chromium (например, *Google Chrome* или *Яндекс Браузер*) со встроенным инструментарием разработчика *DevTools*. Инструментарий используется для инспектирования DOM-дерева, профилирования сетевых запросов к СУБД и анализа логов консоли JavaScript.
- **Инструментарий сборки нативного пакета:** Доступ к веб-интерфейсу специализированной облачной платформы автоматизированной компиляции *AppsGeyser*. Данный сервис применяется для оборачивания веб-дистрибутива в нативный мобильный пакет (APK) для ОС Android без необходимости развертывания локальной тяжелой среды Android Studio.

2. АРХИТЕКТУРА И СТРУКТУРА ПРОГРАММНОГО ПРОДУКТА

2.1. Компоновка исходного кода

В целях обеспечения максимальной производительности, высокой скорости загрузки элементов на мобильных устройствах и упрощения процедуры поддержки программный код приложения реализован в рамках компактной монолитной архитектуры. Весь исполняемый код и интерфейсные решения консолидированы в едином корневом файле **index.html**.

Внутри данного файла выделяются три логических уровня, бесшовно интегрированных друг с другом:

1. **Интерфейсный уровень (HTML5):** Отвечает за декларативную разметку графических элементов, структуру экранов, полей ввода данных, кнопок навигации и списков отображения книг.
2. **Стилистический уровень (CSS3):** Описывает визуальное представление приложения. На данном уровне инкапсулированы правила адаптивной верстки (Media Queries), анимационные эффекты перехода между экранами, а также стили графической концепции Material Design, обеспечивающие эргономику интерфейса.
3. **Логический уровень (JavaScript):** Выступает в роли ядра приложения. Данный блок управляет динамическим поведением интерфейса, обрабатывает пользовательские события (клики, ввод текста), координирует локальные состояния приложения и осуществляет асинхронный обмен данными с внешними серверами экосистемы Firebase.

3. ИНТЕГРАЦИЯ И КОНФИГУРИРОВАНИЕ ОБЛАЧНОЙ ПЛАТФОРМЫ FIREBASE

Для обеспечения персистентного (постоянного) хранения данных и безопасной авторизации пользователей приложение интегрировано с облачной BaaS-платформой Firebase от компании Google. Процесс конфигурирования связи серверной и клиентской частей состоит из следующих последовательных этапов.

3.1. Инициализация проекта в Firebase Console

1. Разработчик осуществляет авторизацию в веб-интерфейсе *Firebase Console* с использованием учетной записи Google.
2. В панели управления создается новый проект под уникальным идентификатором, для которого активируются необходимые сервисы: *Firebase Authentication* (для управления учетными записями читателей) и облачная база данных (например, *Realtime Database* или *Cloud Firestore*).
3. В настройках проекта регистрируется новое веб-приложение (Web App), в результате чего сервер автоматически генерирует уникальный объект конфигурации — параметры SDK.

3.2. Настройка клиентской части (Инициализация `firebaseConfig`)

Полученные параметры авторизации приложения на сервере жестко прописываются внутри клиентского кода. В файле `index.html` внутри управляющего тега `<script>` объявляется константный объект конфигурации `firebaseConfig`:

```
const firebaseConfig = {  
  apiKey: "УНИКАЛЬНЫЙ_КЛЮЧ_ДОСТУПА_API",  
  authDomain: "ИДЕНТИФИКАТОР_://firebaseapp.com",  
  databaseURL: "https://ИДЕНТИФИКАТОР_://firebaseio.com",  
  projectId: "ИДЕНТИФИКАТОР_ПРОЕКТА",  
  storageBucket: "ИДЕНТИФИКАТОР_://appspot.com",  
  messagingSenderId: "ЦИФРОВОЙ_ИДЕНТИФИКАТОР_ОТПРАВИТЕЛЯ",  
  appId: "УНИКАЛЬНЫЙ_ИДЕНТИФИКАТОР_ПРИЛОЖЕНИЯ"  
};
```

Далее с помощью вызова системных методов SDK Firebase (например, `firebase.initializeApp(firebaseConfig)`) происходит регистрация экземпляра приложения и устанавливается защищенный TLS-канал связи с сервером.

3.3. Разграничение прав доступа и правила безопасности

Для защиты конфиденциальных данных пользователей в соответствии со стандартами безопасности и требованиями законодательства РФ (152-ФЗ), в консоли Firebase настраиваются серверные правила безопасности (*Firebase Security Rules*).

Настройки конфигурации правил жестко разграничивают доступ к операциям чтения и записи данных в СУБД. Базовое правило предписывает закрыть прямой неавторизованный доступ к данным из внешней сети, разрешая транзакции только для тех пользователей, которые успешно прошли процедуру верификации через модуль *Firebase Authentication*:

```
{  
  "rules": {  
    ".read": "auth != null",  
    ".write": "auth != null"  
  }  
}
```

Данная конфигурация гарантирует, что каждый пользователь приложения «Книжный трекер» имеет доступ на чтение и модификацию исключительно своей персональной библиотеки и личных заметок, изолируя информацию от стороннего вмешательства.

4. ПОРЯДОК СБОРКИ И ПОДГОТОВКИ ПРОГРАММЫ К РАБОТЕ

Преобразование исходного веб-дистрибутива в нативный мобильный пакет для ОС Android осуществляется через облачную платформу автоматизированной сборки AppsGeyser по следующему алгоритму:

1. **Архивация исходных файлов:** Файл `index.html` (вместе с сопутствующими медиаресурсами, если они используются) помещается в корневую директорию и упаковывается в стандартный архив формата `.zip`.
2. **Загрузка в сервис сборки:** В веб-интерфейсе платформы *AppsGeyser* выбирается тип создаваемого приложения «HTML-код» (или «Веб-сайт»). Созданный `.zip`-архив загружается на сервер через форму импорта.
3. **Конфигурирование метаданных:** В панели настроек AppsGeyser программист указывает манифестные данные приложения: официальное наименование («Книжный трекер»), версию пакета (1.0.0), а также загружает графическую иконку приложения в формате `.png`.
4. **Компиляция:** По нажатию кнопки «Build» (Собрать) облачный сервер выполняет автоматическую компиляцию проекта. На выходе генерируется готовый к установке исполняемый файл с расширением `.apk`, доступный для скачивания.

5. МЕТОДЫ ПРОВЕРКИ РАБОТОСПОСОБНОСТИ ПРОГРАММЫ

Для верификации корректности настройки среды и проверки интеграции с базой данных Firebase необходимо выполнить следующие контрольные действия:

1. **Проверка инициализации SDK:** Запустить файл `index.html` в браузере Google Chrome и открыть консоль разработчика (клавиша F12 -> вкладка *Console*). Отсутствие критических ошибок (Uncaught Errors) и предупреждений со стороны скриптов Firebase свидетельствует о правильности заполнения объекта `firebaseConfig`.
2. **Тестирование канала связи (Авторизация):** На интерфейсном уровне приложения выполнить тестовую регистрацию нового пользователя.

Успешный переход на главный экран приложения подтверждает корректность работы модуля *Firebase Authentication*.

3. **Верификация транзакций СУБД:** Добавить тестовую книгу в личный список приложения. После выполнения операции необходимо открыть административную консоль *Firebase Console* во вкладке базы данных и убедиться, что в дереве данных появилась соответствующая запись с заполненными полями (название книги, автор, дата прочтения, жанр, оценка).